

StickyKV — KV-Cache Quantization Design

Windowed eviction · Two-tier int4 quantization · Interleaved position map · Pre-RoPE ledger

Plain-English walkthrough

Forward pass explained

Colored reference diagrams

THE WHOLE THING IN PLAIN ENGLISH

Okay so here is the entire design without the jargon, written the way I'd explain it to someone over coffee.

The problem

When the model reads a long prompt and starts generating, it remembers every token it has seen so far — that memory is the KV cache. The thing keeps growing and eventually eats the whole GPU. So we have to throw stuff away. The whole game is throwing away the right stuff and keeping the stuff that matters.

Windows

We chop the sequence into fixed-size chunks called windows (say 32 tokens each). Every `window_size` steps we pause, look at how much attention each window has been pulling, and rank them. A couple of windows never get ranked — the sink (first few tokens) and the local window (the most recent one) are always kept, because they always matter.

Three buckets instead of two

Normally a window is either kept or deleted. We add a middle bucket. The best windows stay in full precision fp16 — that's the **K tier**. The ones that aren't good enough for fp16 but are still too useful to throw away, we squeeze down to int4, a quarter of the memory — that's the **Q tier**. Everything else gets dropped. So a window can be kept-full, kept-small, or deleted.

What happens at every eviction (the press cycle)

1. We rank all the windows.
2. The windows crossing into the Q tier get quantized — but right before we quantize, we strip RoPE off them. RoPE is basically the position stamp on a token. We store the stripped, un-stamped version. This pre-RoPE bit is the most important decision we made and I'll explain why in a second.
3. The survivors get squished together so there are no gaps, and we renumber their positions starting from zero.
4. Here is the part that bit us. The renumbering has to cover **both tiers at once**. Say window 1 and window 5 stay in fp16, and window 3 is sitting between them in the Q tier. We can't just renumber 1 and 5 and tack 3 on at the end — that would tell the model window 3 is the newest thing it just saw, which is a lie. So we sort all the survivors back into their original order, lay out one shared set of positions across both tiers together, and the fp windows simply leave a gap in position space where the Q windows live.
5. **And every single press cycle we go back and update the `position_range` of every Q window in the ledger.** The codes never change. The scale never changes. Only this one position number changes. It's one cheap integer write per window. This is the thing we must never forget to do — if the ledger position goes stale, the window gets stamped to the wrong place at read time and the math is wrong.

Why we store them un-stamped (pre-RoPE)

Because we renumber positions on *every* eviction. If we had baked the position stamp into the quantized codes, then every cycle we'd have to un-stamp them, re-stamp them at the new position, and re-quantize — and re-quantizing keeps piling on a little error each time until the window turns to mush. By storing them un-stamped and only stamping fresh when we actually read them, the codes are frozen forever and never drift. The position lives in the ledger as a plain number, not inside the data.

The forward pass — how we actually read it back

When the model needs to attend during generation:

1. The fp16 windows are already stamped, so they're ready to go.
2. For each Q window we look it up in the ledger, grab the int4 codes, blow them back up to fp16 (dequantize), and stamp them with RoPE using the `position_range` we've been keeping fresh.
3. We glue the fp16 windows and the freshly-stamped Q windows into one tensor and hand it to normal attention.
4. The order we glue them in doesn't matter at all. Each key already carries its correct position inside its own values (we just stamped it), so attention works out the right distances no matter where a key physically sits in the tensor.

In Phase 1 we do this the dumb-simple way — blow up the whole Q tier to fp16, glue, attend. In Phase 2 we get clever: we do the blow-up one window at a time *inside* the attention kernel itself, so the fp16 version never even gets written to memory. Only the small int4 version ever lives in HBM, and the dequant + stamp + multiply all happen in registers.

That's the whole thing

Cut into windows, rank them, keep the best in fp16, squeeze the middle into int4, drop the rest. Renumber everyone together every cycle. Keep the ledger's position numbers fresh on every press. And stamp the positions back on only at the moment we read.

1 · TOKEN WINDOWS



Every `window_size` generation steps an eviction fires. Sink (first 4 tokens) never touched. Local window always kept. All others compete on cumulative attention score.

2 · CUMULATIVE H2O SCORING

Attention matrix
Flash: aux SDPA from cached K + recomputed Q. Eager: read `attn_weights`. Shape `[B,H,T,S]`.

→

Sum over query rows
Each query row counts equally. `attn.sum(-2)`. Strip sink, reduce by window.

→

Accumulate in-place
`window_scores += new` every step. Prefill chunked to bound memory.

3 · EVICTION CYCLE (EVERY WINDOW_SIZE STEPS)

① Select	<code>compute_retain_window_indices()</code> Mean scores across heads <code>[B,H,W]→[B,W]</code>. Top-K on evictable slice, sorted chronologically, local windows appended.
② Demote	<code>Rank → Q tier or drop</code> Below-K-above-Q windows demoted: un-rotate once, quantize, write codes + pinned scale/zero to Q store. Below-Q windows dropped.
③ Compact	<code>state.slice_and_keep()</code> <code>torch.gather</code> on fp K/V tensors. Survivors become a contiguous tensor <code>[B,H_kv,T_fp,D]</code>.
④ Position map	<code>build_interleaved_position_map()</code> ← new Sort ALL survivors (fp + Q) by <code>original_window_id</code>. Assign one contiguous <code>arange(T_total)</code> across both tiers. Returns fp target positions and Q-tier <code>position_range</code> assignments.
⑤ Rerotate	<code>rerotate_keys(rope, old_pos, new_pos)</code> Fp keys re-rotated to their interleaved targets (may skip over Q slots). Un-rotate old → apply new.
⑥ Ledger	<code>Update position_range every cycle</code> Each surviving Q window's <code>position_range</code> rewritten from the map. No re-quant — codes are pre-RoPE. Override <code>cache_position</code> to <code>T_total</code>.

W1(fp), W3(Q), W5(fp) survive. W2 and W4 dropped. `window_size = 4` .

✗ **Fp-only compaction (wrong)**

fp_store	sink	W1-0	W1-1	W1-2	W1-3	W5-4	W5-5	W5-6	W5-7
concat append	W3-8		W3-9		W3-10		W3-11		

W3 lands at 8–11 — newer than W5. Model thinks W3 is the most recent context. Relative distances corrupted.

✓ **Interleaved position map (correct)**

sort by id	W1 fp	→	W3 Q	→	W5 fp	→ assign positions 0..11							
full pos space	sink	0	1	2	3	4	5	6	7	8	9	10	11
fp_store rotated	sink	W1-0	W1-1	W1-2	W1-3	—	—	—	—	W5-8	W5-9	W5-10	W5-11
ledger update	W3	position_range ← (4, 7) · codes unchanged · scale/zero unchanged											
at read	dequant W3	→ RoPE at (4,7) → concat [fp W3] — physical order irrelevant											

5 · THE Q-TIER LEDGER (PER-WINDOW RECORD)

One record per Q window, keyed by `original_window_id` . The physical Q store is a bag of bytes — the ledger makes it navigable.

FIELD	FROZEN?	SET WHEN	PURPOSE
<code>original_window_id</code>	yes	birth (prefill)	Chronological identity; sorts both tiers for the interleaved map
<code>codes</code> (int4)	yes	first demotion	Packed quantized bits; never touched again
<code>scale</code> , <code>zero</code>	yes	first demotion	Pinned affine grid; <code>quant(dequant(c))=c</code> exactly, forever
<code>offset</code>	no	each eviction	Byte offset in Q store; shifts as windows drop and the store compacts
<code>position_range</code>	no	each eviction	Slot in the interleaved map; drives RoPE at read. One integer write, no re-quant

6 · TWO-TIER BUDGET

K+ Tier — Full Precision				Q Tier — int4			
Sink + Top-K + Local windows				Demoted high-scoring windows			
Format	fp16 / bf16			Format	int4 + pinned scale/zero		
RoPE	post-RoPE, re-rotated each eviction			RoPE	pre-RoPE → applied at read		
Memory	(1-q) × M_budget			Memory	q × M_budget (≈ ¼ bytes)		

✗ Post-RoPE storage

Every eviction:

dequant → un-rotate → re-rotate → re-quant . Re-quantizing against a pinned grid after values changed is not idempotent → error accumulates each cycle.

✓ Pre-RoPE storage (our design)

At demotion: un-rotate once → quantize → freeze codes. **Each eviction:** update ledger `position_range` only. **At read:** dequant → RoPE at current range. Codes never change → zero drift.

8 · FORWARD PASS — READ PATH EACH STEP

fp_store K/V already rotated + int4 codes → dequant pinned scale/zero → apply RoPE at position_range → concat [fp || Q] order-free → attention

Phase 1: Materialize the full Q store to fp16, concat, standard attention. Transient fp16 freed after each layer.

Phase 2 (Triton): One tile = one window. Load int4 → unpack in registers → apply RoPE arithmetically → MAC. Zero fp16 write-back to HBM.

Why order-free: each key's RoPE rotation already encodes its logical position from the interleaved map, so physical tensor order does not affect attention correctness.

9 · WHAT'S NOVEL

Compact-and-rerotate

KVPress methodology. Every eviction, fp survivors re-indexed to contiguous positions and re-rotated. Query position overridden to match.

Interleaved position map

Both tiers sorted by `original_window_id` and positioned jointly. Fp keys skip over Q slots; Q ledger gets the right chronological slot.

Pinned-grid pre-RoPE int4

Codes quantized once, stored un-rotated, frozen. Only `position_range` changes each cycle — a free integer write. Zero rope-driven drift.

Write-once ledger

Only structure tracking Q windows across compactions. Two frozen fields, two mutable. Keeps the Q store a dense gap-free byte array.

Tile-fused dequant

Phase 2: one Triton tile = one window = one scale group. Dequant + RoPE + MAC in registers. No cross-tile bookkeeping, no fp16 write-back.

Cumulative H2O scoring

Scores accumulate in-place via sum-over-query-rows. Prefill chunked to avoid the full `[B,H,T,S]` matrix. Feedback drift watched via Suite A Jaccard.